Reviewer Pack — Minimal Order of Geometric Galois Groups and DPLL Proof Tree…

Entry 1fe1848085ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 18:54:44 UTC

Minimal Order of Geometric Galois Groups and DPLL Proof Tree Height Inequality

Entry ID: `1fe1848085ac` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 18:54:44 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Geometric Galois Groups) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

> For every Boolean satisfiability problem φ, the minimal order of its geometric Galois group is linearly correlated with its DPLL proof tree height, such that ord(G(φ)) = Ω(log w(φ)), where G(φ) is the geometric Galois group and w(φ) is the DPLL proof tree height.

Rationale (proposer's reasoning):

> Geometric Galois groups provide a bridge between algebraic geometry and topological properties of spaces. Their minimal orders can reflect the complexity inherent in the problem structure, which could potentially expose hidden connections with computational complexity like the DPLL proof tree height that measures the difficulty of solving SAT instances.

Taxonomy category: `GeometricGroupTheoryToDPLLProofComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cdd833787c55a556`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 independent runs with varying seeds, the mean order of G(φ) for all φ satisfies ord(G(φ)) ≥ log w(φ) for at least 80% of instances.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(i, n + 1):
                A[i][j] /= factor
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(i, n + 1):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m = len(A)
        p = len(B[0])
        q = len(B)
        C = [[0 for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(q):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def determinant(A):
        n = len(A)
```

```
        if n == 1:
            return A[0][0]
        det = 0
        sign = 1
        for i in range(n):
            minor = [row[:i] + row[i+1:] for row in A[1:]]
            det += sign * A[0][i] * determinant(minor)
            sign *= -1
        return det

    def geometric_galois_group_size(A):
        n = len(A)
        if n == 0:
            return 1
        rank = sum(1 for row in gaussian_elimination(A) if any(row))
        return math.factorial(n) // (math.factorial(rank) * math.factorial(n - rank))

    def dpll_proof_tree_height(phi):
        # Placeholder function to simulate DPLL proof tree height calculation
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, 10)

    instances_tested = 0
    n_max = 0
    total_order = 0
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        phi = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        order = geometric_galois_group_size(phi)
        height = dpll_proof_tree_height(phi)

        if instances_tested == 0:
            n_max = n

        total_order += order
        instances_tested += 1

    mean_order = total_order / instances_tested
    conjecture_holds = mean_order >= math.log(n_max)

    return {
        "metric_name": "geometric_galois_group_size",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample if not conjecture_holds else ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
```

```
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if not trial_result["conjecture_holds"]:
            counterexample = trial_result["counterexample"]
            break
        else:
            results.append(trial_result)

    mean_order = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[re
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b329e5.py", line 108, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b329e5.py", line 81, in run_trial
    order = geometric_galois_group_size(phi)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b329e5.py", line 66, in geometric_gal
    rank = sum(1 for row in gaussian_elimination(A) if any(row))
                            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b329e5.py", line 31, in gaussian_elim
    A[i][j] /= factor
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17010
2	preregistration	ollama_remote	glm4:latest	0	0	9255
3	novelty	ollama_remote	glm4:latest	0	0	8544
4	novelty	ollama_remote	glm4:latest	0	0	8746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15387
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13506
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13200
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11872
9	judge	ollama_remote	glm4:latest	0	0	14453

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111971 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1fe1848085ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1fe1848085ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1fe1848085ac.tar.gz` (if generated)